Fluffy the Squirrel is starting a restaurant, and he is creating the menu. The menu currently has just one *layer*, which is the Food *layer*.

- Fried Rice
- Fried Noodle
- Water

Fluffy is a good chef, and allows for more *layers* of options. For example, he could add a *layer* for Portion Size with two options, Small and Large. He will expand the menu by expanding each Food *layer* item with the sub-list containing options for Portion Size *layer*, like so:

- Fried Rice
    - Small
    - Large
- Fried Noodle
    - Small
    - Large
- Water
    - Small
    - Large

Every time he adds on another *layer*, he will take each item of the **innermost** *layer*, and add a new sub-list to each of them, and the newly added *layer* will be the new innermost *layer*. So in this case, if he were to add on another *layer*, he will expand each of the Small's and Large's (because the innermost layer now is the Portion Size) with a new sub-list.

If you are still confused as to how the menu is expanded, please refer to Appendix.

Looking at the above menu, Fluffy counted nine lines. However, he realised that if he changed the order of *layers* to start with Portion Size instead of Food, the menu will look like

- Small
    - Fried Rice
    - Fried Noodle
    - Water
- Large
    - Fried Rice
    - Fried Noodle

- Water

which only takes eight lines, and the menu is still functional! (even though it might look unorthodox)

Fluffy is lazy and hence doesn't like to write too much, so he wonders given some layers and their sizes, how can he arrange the layers so that it minimises the number of lines in the menu?

Input Format

Suppose there are N layers, the input will be an integer N and an array `layers` of N integers, the i^{th} number represents the size of the i^{th} layer.

Constraints:

- $1 \leq N \leq 30.$
- $1 \leq$ `layer[i]` $\leq 10^4$ for all valid `i`

Output Format

There are two types of cases for this problem.
For cases 1 - 5, output N space-separated integers, representing the sizes of the *layers* after the rearrangement that gives the minimum number of lines.
For cases 6 - 10, find the minimum number of lines. Since the number of lines may be huge, output the remainder when it is divided by $10^9 + 7$.

Hint: Be careful with big integers in the larger cases! Some programming languages like C++, Java have limits for their integer datatypes.
You should either use something with no limits for their integers like Python, or use the simple fact that `(a + b) % r = ((a % r) + (b % r)) % r` and/or `(a * b) % r = ((a % r) * (b % r)) % r` where `%` is the remainder operation.

Sample Input

```
N = 2
layers = [3, 2]
```

Sample Output

If the sample input is among task 1 - 5, the output should be

```
2 3
```

If the sample input is among task 6 - 10, the output should be

```
8
```

Explanation

This sample corresponds to the example in the problem statement above. The optimal arrangement starts with size `2`, followed by size `3`.

Appendix

This section is to illustrate further how the menu is actually being built. We believe the above statement is sufficiently clear but here is one more example with step-by-step explanation. Suppose we want to built a menu with 3 *layers* called A, B, and C respectively. *Layer* A has two options, A1 and A2, *layer* B has two options, B1 and B2, and *layer* C has two options, C1 and C2. Suppose we have decided to order of the layers to be A $\rightarrow$ B $\rightarrow$ C

Starting with *layer* A, gives us the menu

- A1
- A2

Currently the innermost *layer* is A.

Next, we need to add on the *layer* B. We take each of the innermost layer's item (the A1 and A2) and add the *layer* B sub-list, giving us

- A1
  - B1
  - B2
- A2

    - B1
    - B2

So now, the new innermost *layer* is B.

Finally, we will add on the *layer* C. We take each of the innermost layer's item (the B1's and B2's) and add the *layer* C sub-list, giving us

- A1
    - B1
        - C1
        - C2
    - B2
        - C1
        - C2
- A2
    - B1
        - C1
        - C2
    - B2
        - C1
        - C2

This gives us a total of 14 lines.

Fluffy sang tupai ingin membuka suatu restoran, dan dia sedang membina menunya. Setakat ini, menu tersebut hanya mempunyai satu *lapisan*, iaitu *lapisan* Food:

- Fried Rice
- Fried Noodle
- Water

Fluffy adalah seorang chef yang baik, dan dia boleh menambahkan *lapisan* pilihan. Sebagai contoh, dia boleh menambahkan *lapisan* bagi Portion Size dengan dua pilihan, Small dan Large. Dia boleh mengembangkan menu tersebut dengan mengembangkan setiap perkara dalam *lapisan* Food dengan senarai pilihan bagi *lapisan* Portion Size, seperti berikut:

- Fried Rice

    - Small
    - Large
- Fried Noodle
    - Small
    - Large
- Water
    - Small
    - Large

Setiap kali dia menambahkan satu *lapisan* baru, dia mengambil setiap perkara dalam *lapisan* paling **dalam**, dan menambahkan senarai yang baru kepada setiap perkara tersebut, dan *lapisan* yang baru ditambah tersebut akan menjadi *lapisan* paling dalam yang terbaru. Dalam kes ini, jika dia menambahkan satu lagi *lapisan*, dia akan mengembangkan setiap perkara Small dan Large (kerana *lapisan* paling dalam ialah Portion Size), dengan senarai yang baru.

Jika anda masih keliru mengenai pembinaan menu tersebut, rujuk kepada ruangan Appendix.

Melihat kepada menu di atas, Fluffy mengira sejumlah sembilan baris. Walau bagaimanapun, dia menyedari bahawa jika dia menukarkan turutan *lapisan* untuk bermula daripada Portion Size dan bukannya Food, menu tersebut akan menjadi seperti

- Small
    - Fried Rice
    - Fried Noodle
    - Water
- Large
    - Fried Rice
    - Fried Noodle
    - Water

yang hanya memerlukan lapan baris, dan menu tersebut masih berfungsi! (walaupun ia mungkin kelihatan janggal).

Fluffy adalah seorang yang pemalas dan tidak suka untuk menulis panjang-panjang, jadi beliau berfikir bahawa jika diberi beberapa lapisan dan saiz setiap lapisan, bagaimanakah untuk beliau menyusun lapisan tersebut untuk meminimumkan bilangan baris dalam menu tersebut?

Format Input

Andaikan bahawa wujud N lapisan, input adalah suatu integer N dan suatu tatasusunan (array) `layers` yang terdiri daripada N integer, di mana integer ke-i mewakili saiz bagi lapisan ke-i.

Kekangan:

- $1 \le N \le 30$.
- $1 \le$ `layer[i]` $\le 10^4$ untuk semua `i` yang sah

Format Output

Terdapat dua jenis kes ujian (test case) bagi masalah ini.
Bagi kes ujian 1 - 5, output N integer yang diasingkan dengan satu ruang. N integer ini mewakili saiz *lapisan* selepas disusun dengan bilangan baris yang minimum.
Bagi kes ujian 6 - 10, cari bilangan baris yang minimum. Oleh kerana bilangan baris mungkin sangat besar, output bakinya apabila dibahagi dengan $10^9 + 7$.

Petunjuk: Berhati-hati dengan integer besar dalam kes yang lebih besar! Beberapa bahasa pengaturcaraan seperti C++, Java mempunyai had saiz integer. Anda dinasihati untuk menggunakan sesuatu tanpa had saiz integer seperti Python, atau menggunakan kesamaan berikut: `(a + b) % r = ((a % r) + (b % r)) % r` dan `(a * b) % r = ((a % r) * (b % r)) % r`, dengan `%` sebagai operasi baki.

Contoh Input

```
N = 2
layers = [3, 2]
```

Contoh Output

Jika contoh input adalah daripada tugasan 1 - 5, outputnya ialah

```
2 3
```

Jika contoh input adalah daripada tugasan 6 - 10, outputnya ialah

```
8
```

Penjelasan

Contoh ini adalah serupa dengan contoh dalam pernyataan soalan di atas. Susunan yang bagi bilangan baris minimum bermula dengan saiz `2`, dan selepas itu saiz `3`.

Appendix

Bahagian ini bertujuan untuk menghuraikan dengan lebih jelas mengenai cara pembinaan menu tersebut. Kami berpendapat bahawa pernyataan yang diberikan sudah cukup jelas tetapi kami akan memberikan satu lagi contoh beserta penjelasan langkah demi langkah.
Andaikan kita mahu membina suatu menu dengan 3 *lapisan* yang masing-masing dipanggil A, B, dan C. *Lapisan* A mempunyai dua pilihan, A1 dan A2, *lapisan* B mempunyai dua pilihan, B1 dan B2, dan *lapisan* C mempunyai dua pilihan, C1 dan C2.
Andaikan kita menyusun lapisan tersebut mengikut urutan A $\rightarrow$ B $\rightarrow$ C.

Bermula dengan *lapisan* A, kita mendapat menu

- A1
- A2

Setakat ini, *lapisan* paling dalam ialah A.

Kemudian, kita tambahkan *lapisan* B. Kita ambil setiap perkara daripada lapisan paling dalam (iaitu A1 dan A2) dan tambahkan pilihan dari *lapisan* B, untuk mendapatkan

- A1
  - B1
  - B2
- A2
  - B1
  - B2

Maka sekarang, *lapisan* paling dalam ialah B.

Akhir sekali, kita tambahkan *lapisan* C. Kita ambil setiap perkara daripada lapisan paling dalam (iaitu B1 dan B2) dan tambahkan pilihan dari *lapisan* C, untuk mendapatkan

- A1
  - B1

        - C1
        - C2
    - B2
        - C1
        - C2
- A2
    - B1
        - C1
        - C2
    - B2
        - C1
        - C2

Ini memberikan kita sejumlah 14 baris.

Solution

Let the sequence be A. By experimenting, we notice that the number of lines is given by $A_1 + A_1A_2 + A_1A_2A_3 + \ldots + A_1A_2 \ldots A_N$. We can intuitively see that we get to reduce the number of lines most effectively by reducing A_1, followed by reducing A_2, and so on. Therefore, the optimal order is the increasing order.

Indeed, we can formally prove this. Suppose A is not sorted in increasing order, then there exist indices i and j ($i < j$) such that $A_i > A_j$.

Let

$$X = A_1 + \ldots + A_1 \ldots A_{i-1}$$
$$Y = A_1 \ldots A_j + \ldots + A_1 \ldots A_N$$
$$Z = 1 + A_{i+1} + A_{i+1}A_{i+2} + \ldots + A_{i+1} \ldots A_{j-1}$$

Then,

$$
\begin{aligned}
&A_1 + \ldots + A_1 \ldots A_N \\
&= X + A_1 \ldots A_i + \ldots + A_1 \ldots A_{j-1} + Y \\
&= X + Y + (A_1 \ldots A_i)(1 + A_{i+1} + A_{i+1}A_{i+2} + \ldots + A_{i+1} \ldots A_{j-1}) \\
&= X + Y + (A_1 \ldots A_i) \cdot Z
\end{aligned}
$$

Since X and Z do not involve A_i and A_j and all terms in Y contain both A_i and A_j, swapping the values of A_i and A_j does not change their values. However, since $A_i > A_j$, swapping them does decrease $A_1 \ldots A_i$, which implies that A is not an optimal ordering.

The biggest challenge of this problem is calculating the number of lines in modulo $10^9 + 7$, but since n is small, any reasonable solution should pass.

Alternatively, some may also find the equivalent formula $A_1(1 + A_2(1 + A_3(1 + \ldots (1 + A_N) \ldots)))$ instead. One way to compute this formula is to compute starting from the middle or use recursion.